AI101: Data Science and AI Essentials with Python | School of AI and Data Engineering, IIT Ropar

SMS Spam Detection

using Logistic Regression

Mini-Project (team of 4) | Team no.: [TEAM NUMBER]

Member	Entry no.	GitHub handle	Role
[Name 1]	[Entry no.]	[Handle]	Documentation Lead
[Name 2]	[Entry no.]	[Handle]	Execution Lead
[Name 3]	[Entry no.]	[Handle]	PPT Lead
[Name 4]	[Entry no.]	[Handle]	Video Lead

Repo: https://github.com/Obliterator-Abhinav/sms-spam-detection

Video: [VIDEO LINK]

Problem and Data

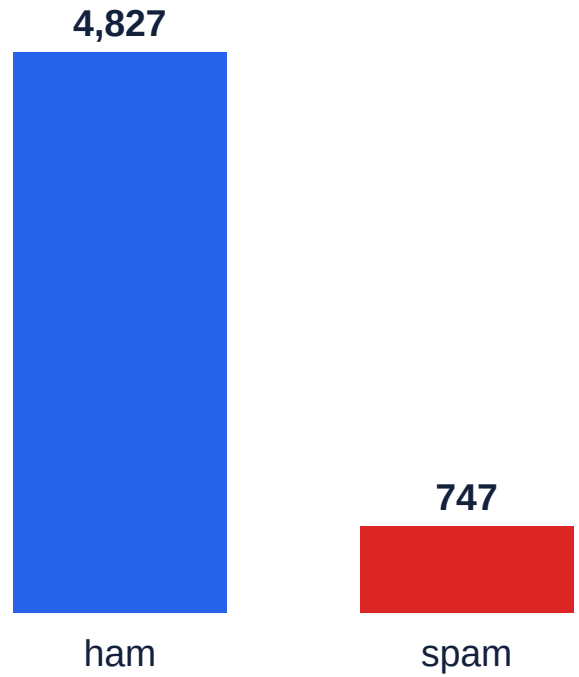
The problem

- Decide if an SMS is spam (unwanted ad or scam) or ham (a normal message).
- A filter must block scams without blocking real messages. A wrongly blocked real message is usually worse than one spam getting through.
- Binary classification: spam = 1 (positive class), ham = 0. Model: Logistic Regression.

The data

- SMS Spam Collection, UCI ML Repository (CC BY 4.0). Almeida & Hidalgo (2011).
- 5,574 messages, 2 columns: label and message. No missing values.
- 403 duplicates removed, leaving 5,171 (4,518 ham, 653 spam).
- Extra numeric features: message length, digit count, capital-letter ratio, exclamation marks.

Class balance (original data)

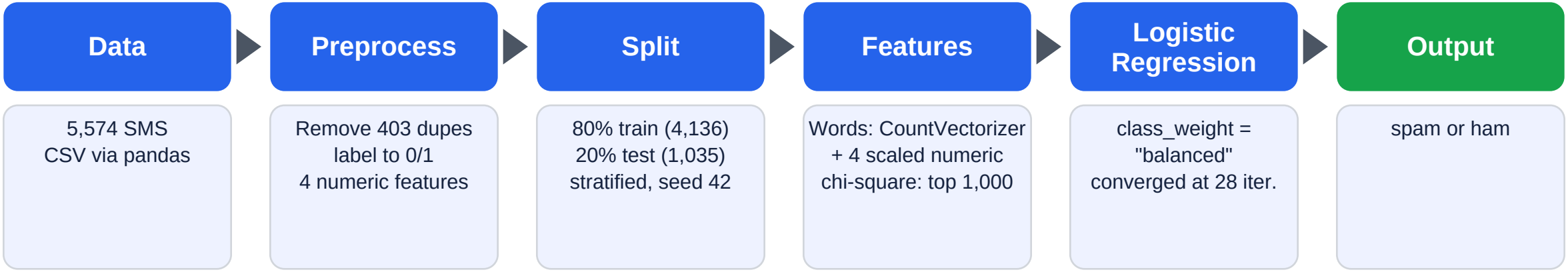


About 13% spam. Always saying "ham" gives ~87% accuracy, so accuracy alone is misleading.

Approach

Pipeline

Scaler and vectoriser are fitted on training data only.

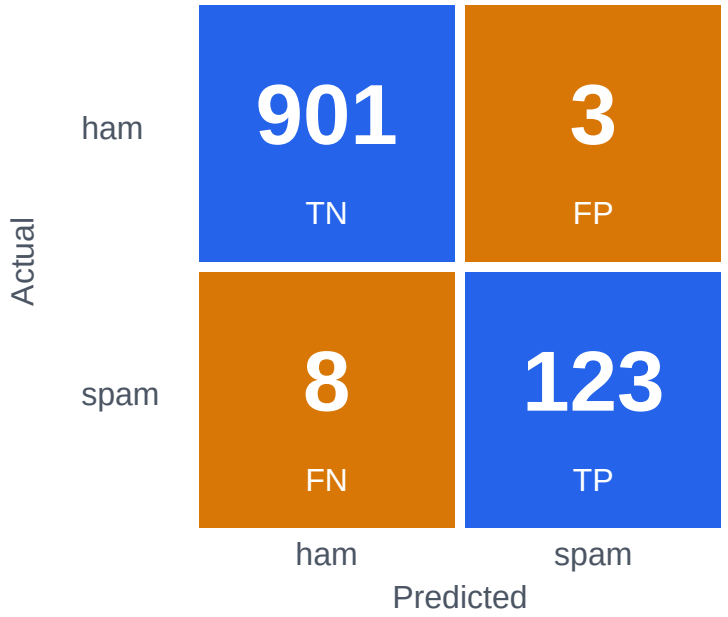


Final input and libraries

- 1,004 columns per message: 1,000 words + 4 numeric features (3,310 words before selection).
- Python 3.12 / 3.13, pandas, numpy, scikit-learn, scipy, matplotlib, seaborn, Jupyter Notebook.

Results

Confusion matrix (test set, 1,035 messages)



0.9894	0.9762	0.9389	0.9572
Accuracy	Precision (spam)	Recall (spam)	F1-score (spam)

- Only 3 real messages wrongly sent to spam (false positives), which matters most for a spam filter.
- Found 123 of 131 spam messages; missed 8.
- Train F1 0.983 vs test F1 0.957: small overfitting, test results still strong.
- Balanced class weights raised recall from 0.908 to 0.939.

[DEMO SCREENSHOT]

docs/expected_output.png

Reproduce: pip install -r requirements.txt, then run SMS_Spam_Logistic_Regression.ipynb (Run All Cells).

Learnings

What worked

- Word counts + 4 numeric features.
- digit_count was the strongest signal (coef. 3.10; spam averages ~15 digits, ham ~0.3).
- Balanced class weights.
- High precision: only 3 false positives.

What is limited

- Missed 8 spam: short or normal-looking texts, one with scrambled spelling.
- Dataset quirks: "lt"/"gt" come from <#> in 197 ham messages; "arsenal" learned from 2 spam messages.
- Data is from ~2011 and in English; today's Indian spam (OTP, Hinglish) not tested.
- Settings not tuned; word counts ignore order.

Who did what

- [Name 1]: Documentation Lead - [what they built]
- [Name 2]: Execution Lead - [what they built]
- [Name 3]: PPT Lead - [what they built]
- [Name 4]: Video Lead - [what they built]
- All four members contributed code (see commit history).